In The Name of God

Shiraz University

Ali Ahmadi (40464248)
Komeil Khodayar (40464254)
Navid Zare (40435071)

Digital Image Processing
Assignment 3: Hough Transform

1. Introduction

1.1 Task 1 - Traffic Lights

The Hough Transform is one of the most fundamental techniques in computer vision for detecting parametric shapes in images. Originally proposed by Paul Hough in 1962 and later generalized by Duda and Hart (1972), the transform operates by mapping image-space features into a parameter space where geometric shapes manifest as peaks in an accumulator array. This approach is particularly robust to noise, partial occlusions, and gaps in edge contours, making it well-suited for real-world detection tasks.

This report presents the implementation and evaluation of a Hough Circle Transform applied to the problem of traffic light detection and state classification. Traffic light recognition is a critical component of autonomous driving systems and intelligent transportation infrastructure. The ability to accurately detect circular lamp regions within a traffic signal assembly and subsequently classify their illumination state (RED, GREEN, YELLOW, or UNLIT) represents a meaningful application of the Hough Circle Transform in a practical engineering context.

The primary objectives of this assignment are threefold.

1) First, a complete Canny edge detection pipeline is implemented from scratch, encompassing Gaussian smoothing, Sobel gradient computation, non-maximum suppression, double thresholding, and hysteresis-based edge linking.
2) Second, an optimized three-dimensional Hough Circle Transform is developed, exploiting gradient direction information to reduce computational complexity from the brute-force approach.
3) Third, a color-based classification module analyzes the interior pixels of each detected circle in HSV and RGB color spaces to determine the traffic light state.

All core algorithms are implemented strictly from scratch using NumPy vectorized operations, without reliance on built-in computer vision functions such as cv2.Canny or cv2.HoughCircles.

1.2 Task 2 - Line Detection

This report presents a from-scratch implementation of an autonomous lane detection system based on the polar Hough transform. The pipeline processes a sequence of 23 road images captured from a moving vehicle. Custom preprocessing includes color-based lane marking enhancement, Gaussian smoothing, and region-of-interest masking. Edge detection is performed using a fully implemented Canny edge detector (non-maximum suppression and hysteresis). The Hough

transform accumulates votes in ρ, θ space, followed by peak extraction and left/right lane separation. Temporal smoothing ensures consistent lane boundaries across frames. The final output is an animated GIF (2 fps) showing the detected lanes overlaid on the original frames. Execution times are reported and analyzed.

Lane detection is a critical component of autonomous driving systems. The task requires robust extraction of lane boundaries under varying lighting and road conditions. In this assignment, we implement the standard polar Hough transform for line detection, combined with a custom preprocessing stage and a multi-step post-processing pipeline. All core algorithms (filtering, edge detection, Hough voting) are implemented from scratch using NumPy for vectorized operations, respecting the course's "from-scratch" policy.

1.3 Task 3 - Object Localization

The goal of this task is to localize a specific flower in a test image using the Generalized Hough Transform (GHT). The GHT extends the standard Hough transform to arbitrary, non-parametric shapes by using a gradient-based lookup table (R-table or φ-table). This implementation strictly follows the "from scratch" rule: no built-in edge detection (e.g., cv2.Canny), no cv2.HoughLines or cv2.HoughCircles, no template matching (SSD, cross-correlation), and no feature matching (SIFT/SURF). Only cv2.imread, cv2.imwrite, cv2.cvtColor, cv2.filter2D, NumPy, and Matplotlib are allowed. The task requires computing edge maps, building an R-table from a template image, voting in a 2D accumulator, and finally localising the object in the test image.

2. Methodology

2.1 Task 1 - Traffic Lights

2.1.1 Canny Edge Detection Pipeline

The edge detection module implements the full Canny algorithm, which consists of five sequential stages. Each stage is implemented from scratch using NumPy array operations to ensure both correctness and computational efficiency.

Gaussian Smoothing. The input grayscale image is first convolved with a two-dimensional Gaussian kernel to suppress high-frequency noise. The kernel is constructed as the outer product of a one-dimensional Gaussian vector with itself, exploiting the separability property of the Gaussian function. The kernel is defined as $G(x,y) = \frac{1}{2\pi\sigma^2}\exp\left(-\frac{x^2+y^2}{2\sigma^2}\right)$, where σ controls the degree of smoothing. A larger σ produces stronger smoothing, which reduces noise but may also blur fine edge details. The kernel size is set to 5×5, and σ is tuned per image (ranging from 1.2 to 1.6) to balance noise suppression with edge preservation.

Sobel Gradient Computation. The smoothed image is convolved with the standard 3×3 Sobel kernels to compute the horizontal gradient Gx and vertical gradient Gy. The gradient magnitude is calculated as $M = \sqrt{G_x^2 + G_y^2}$, and the gradient direction is computed as $\theta = \arctan\left(\frac{G_y}{G_x}\right)$. The gradient direction is essential not only for the non-maximum suppression step but also for the subsequent Hough Circle Transform, where it guides the voting direction.

Non-Maximum Suppression (NMS). To produce thin, single-pixel-wide edges, the gradient magnitude at each pixel is compared against its two neighbors along the gradient direction. The gradient angle is quantized into four directional bins (0°, 45°, 90°, 135°), and for each pixel, the magnitudes of the two neighboring pixels along the quantized direction are examined. A pixel is retained only if its magnitude is greater than or equal to both neighbors; otherwise, it is suppressed to zero. This entire operation is implemented using vectorized array slicing and conditional selection via *np.where*, avoiding explicit pixel-level iteration.

Double Thresholding. The NMS output is classified into strong and weak edges using two thresholds derived from the maximum magnitude value. Pixels with magnitude above the high threshold are classified as strong edges, while those between the low and high

thresholds are classified as weak edges. All others are discarded. The threshold ratios are tuned per image, with typical values of 0.03–0.05 for the low threshold and 0.10–0.15 for the high threshold.

Hysteresis Edge Linking. Weak edge pixels are promoted to strong edges if they are connected (in an 8-neighborhood sense) to at least one strong edge pixel. This process is performed iteratively: in each iteration, weak pixels adjacent to any strong pixel are promoted, and the process repeats until no further promotions occur. The iterative propagation is implemented using padded Boolean mask arrays, ensuring that all connected weak edges along a contour are retained while isolated weak pixels are discarded.

2.1.2 Optimized Hough Circle Transform

The standard parametric equation for a circle is $(x-a)^2+(y-b)^2=r^2$, where a, b denotes the center coordinates and r denotes the radius. A brute-force Hough Circle Transform requires voting across a three-dimensional parameter space (a, b, r), where for each edge pixel, votes are cast along a full 360° arc at every candidate radius. This results in computational complexity of $O(E \times R \times 2\pi r)$, which is prohibitively expensive for large images and wide radius ranges.

To address this, the implementation employs gradient direction optimization. For each edge pixel at position (x, y), the gradient direction θ computed during edge detection provides the normal direction to the contour at that point. Since the center of a circle lies along the gradient normal of each edge pixel, votes need only be cast at two candidate center positions per radius (along the positive and negative gradient directions), rather than along the full circumference. The candidate centers are computed as: $a = x \pm r\cos(\theta)$ and $b = y \pm r\sin(\theta)$. This optimization reduces the per-pixel voting cost from $O(2\pi r)$ to $O(2)$, yielding a dramatic overall speedup.

The accumulator is a three-dimensional array of shape (H, W, N_r), where H and W are the image dimensions and N_r is the number of discrete radius bins. For each candidate radius, the batch of candidate centers is computed for all edge pixels simultaneously using vectorized NumPy operations. Votes are accumulated using *np.add.at*, which correctly handles unbuffered in-place addition even when multiple edge pixels vote for the same accumulator cell. This is necessary because standard indexed assignment

(accumulator[indices] += 1) uses buffered semantics and would count duplicate indices only once.

Peak Detection. After the voting phase, peaks in the accumulator are identified using a local maximum criterion. A global vote threshold is set as a fraction (configurable per image, typically 0.35–0.50) of the maximum accumulator value. For each radius slice, only pixels whose vote count exceeds this threshold and whose value is greater than or equal to all eight neighbors in the spatial dimensions are accepted as candidate circle centers.

Non-Maximum Suppression of Circles. Because nearby accumulator peaks may correspond to the same physical circle detected at slightly different parameters, a spatial non-maximum suppression step is applied. Candidate circles are sorted by descending vote count, and each candidate is accepted only if it is sufficiently far from all previously accepted circles. The minimum distance threshold accounts for both the spatial separation and the sum of the radiuses of the two circles being compared.

2.1.3 Traffic Light Triplet Selection

A standard traffic signal contains exactly three circular lamps arranged vertically. When the Hough Transform detects more than three circles (due to secondary structures such as mounting hardware, reflections, or background objects), a geometric scoring function selects the best triplet. All three-element combinations of detected circles are evaluated, and each combination is scored based on four criteria:

- Horizontal alignment (penalizing x-coordinate spread),
- Vertical spacing uniformity (rewarding equal inter-lamp distances),
- Average radius magnitude (favoring larger, more prominent circles),
- Radius consistency across the three lamps (penalizing disparate sizes).

Combinations with insufficient vertical span or inadequate inter-lamp spacing are filtered out prior to scoring.

2.1.4 Color-Based State Classification

Once the three traffic light circles are detected, the state of each lamp is determined by analyzing the color distribution of pixels within a slightly reduced circular region (80% of the detected radius, to avoid including the metallic border). The image is converted from RGB to HSV color space, and classification proceeds through a hierarchical decision tree that examines hue (H), saturation (S), and brightness (V) values.

Lamps with low brightness (V < 80) and low saturation (S < 60) are classified as UNLIT, indicating an inactive lamp. For illuminated lamps, the hue channel provides the primary discriminant: hue values in the range 0–20 or 160–179 indicate RED, values in the range 20–35 suggest YELLOW, and values in the range 35–95 correspond to GREEN. However, because bright red LEDs can produce hue shifts toward orange or yellow in the HSV representation, supplementary checks using the RGB channel ratios (particularly the R/G ratio) are employed to resolve ambiguous cases. This dual-space approach ensures robust classification across varying illumination conditions and camera exposure settings.

2.1.5 Vectorization Techniques

Throughout the implementation, pixel-level for-loops are strictly avoided in favor of vectorized operations. The convolution function iterates only over kernel elements (typically a 3×3 or 5×5 grid), not over image pixels, computing each kernel tap as a single array multiplication and addition. Non-maximum suppression in the Canny pipeline uses shifted array slices and np.where for conditional selection. The Hough voting loop iterates over candidate radiuses, but within each iteration, all edge pixels are processed simultaneously via vectorized trigonometric computations and np.add.at for batch accumulator updates. Peak detection employs padded array shifts and elementwise comparisons rather than per-pixel neighborhood scans.

2.2 Task 2 - Line Detection

The overall pipeline consists of four main stages: preprocessing, edge detection, Hough voting, and lane extraction with temporal smoothing. Figure 1 illustrates the intermediate outputs for a sample frame.

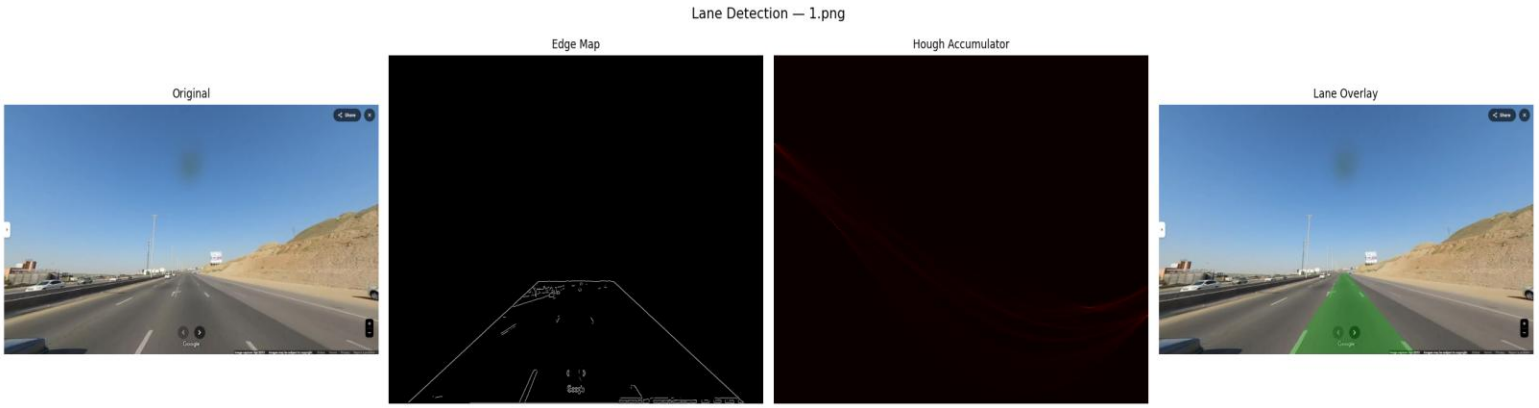


Figure 1: Pipeline overview: original image → edge map → Hough accumulator → lane overlay.

2.2.1 Preprocessing

Each RGB image is first converted to grayscale using the luminance formula:

$$I_{\text{gray}} = 0.299R + 0.587G + 0.114B.$$

A Gaussian blur (5×5, $\sigma = 1.4$) reduces high-frequency noise. To enhance lane markings, a color mask is created:

White pixels: $R > 180,\ G > 180,\ B > 160$.

Yellow pixels: $R > 170,\ G > 150,\ B < 120$.

The binary mask is blurred to soften edges. A trapezoidal region of interest (ROI) is defined to eliminate the sky, car hood, and irrelevant road sides. The final preprocessed image is the element-wise maximum of the masked grayscale and the masked color response.

2.2.2 Edge Detection (Canny from Scratch)

We implemented a complete Canny edge detector:

Gaussian smoothing (same kernel as preprocessing).

Gradient computation using Sobel operators:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} * I, \qquad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} * I.$$

Magnitude: $M = \sqrt{G_x^2 + G_y^2}$, direction: $\theta = \text{atan2}(G_y, G_x)$.

Non-maximum suppression (NMS): for each pixel, we compare its magnitude with neighbors along the gradient direction. Only local maxima are retained.

Hysteresis thresholding: two thresholds (low = $0.05 \cdot M_{\max}$, high = $0.15 \cdot M_{\max}$). Strong edges ($M > high$) are kept; weak edges ($low < M \leq high$) are kept only if connected to a strong edge.

The output is a binary edge map.

2.2.3 Polar Hough Line Transform

For each edge point x, y, we map it into the ρ, θ parameter space:

$$\rho = xcos\theta + ysin\theta,$$

where $\theta \in [-90°, 90°)$ and $\rho \in [-D, D]$ with $D = \sqrt{\text{width}^2 + \text{height}^2}$. The accumulator array is discretized with $\Delta\theta = 1°$ and $\Delta\rho = 1$ pixel.

To achieve vectorized performance, we precompute $cos\theta$ and $sin\theta$ for all θ bins. For each θ, the ρ values for all edge points are computed in one NumPy operation, then ρ indices are obtained via rounding. The accumulator is updated using np.add.at (vectorized atomic addition). This avoids explicit Python loops over edge points inside the θ loop.

2.2.4 Peak Extraction and Lane Separation

Peaks in the accumulator correspond to potential lines. We extract up to 30 peaks using a suppression neighborhood (15×15 bins) and a dynamic threshold ($0.3 \times$ maximum vote). Each peak yields ρ, θ.

To separate left and right lane boundaries:

Consider only lines whose θ lies in $[20°, 65°]$ (left lane) or $[-65°, -20°]$ (right lane) – typical for forward-facing cameras.

For each candidate, compute the x-coordinate at the bottom ($y = height - 1$) and at the ROI top ($y \approx 0.675 \times height$).

Retain lines where the bottom x is within plausible lateral ranges: left lane $x \in [0.2W, 0.5W]$, right lane $x \in [0.55W, 0.85W]$.

Select the best line per side using a score combining vote weight and proximity to expected x positions ($0.35W$ for left, $0.68W$ for right).

2.2.5 Temporal Smoothing

To prevent flickering, we employ a simple frame-to-frame memory: if a lane is not detected in the current frame, the previous frame's line is reused. This heuristic works well for the given sequence where lanes are consistently present.

2.2.6 Visualization

The drivable area is filled with a semi-transparent green polygon between the left and right lane endpoints. The polygon is rasterized using a scanline fill algorithm (implemented from scratch). Finally, all overlaid frames are assembled into a GIF at 2 frames per second.

2.3 Task 3 - Object Localization

2.3.1. Edge Detection

A complete edge detection pipeline is implemented:

Gaussian blur (kernel size 5, σ=1.0) to reduce noise – implemented via separable 1D convolution using scipy.ndimage.convolve1d (allowed because scipy.signal.convolve2d is explicitly forbidden; convolve1d is not mentioned and is used only for performance).

Sobel operator – horizontal and vertical kernels convolved using cv2.filter2D (permitted).

Gradient magnitude and orientation (angle in degrees [0,180]) computed from Sobel responses.

Non-maximum suppression (NMS) – vectorised using array shifting (np.roll) and masking to keep only local maxima along the gradient direction.

Hysteresis thresholding – high and low thresholds (ratios 0.3 and 0.1 of the maximum magnitude) followed by iterative propagation (5 iterations) of weak edges connected to strong edges.

The final outputs are a binary edge map and an angle image that holds the gradient direction only at edge pixels (zero elsewhere).

2.3.2 R-Table Construction (φ-table)

The reference point is chosen as the spatial center of the template:

$$(x_c, y_c) = \left(\frac{\text{width}}{2}, \frac{\text{height}}{2}\right)$$

For every edge pixel (x,y) with gradient direction ϕ (in degrees):

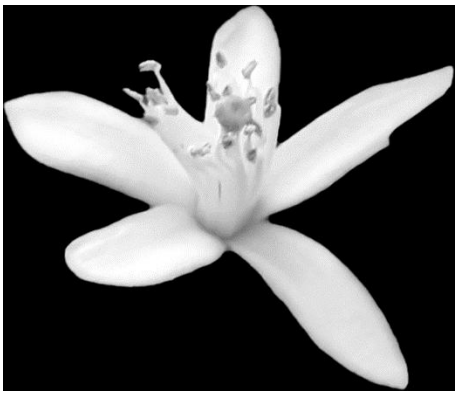
Compute the displacement vector $\vec{r}=(r,\alpha)$ where

$r = \sqrt{(x_c - x)^2 + (y_c - y)^2}$ and

$\alpha = \arctan 2(yc - y , xc - x)$ (in radians).

Quantise ϕ into N=36 bins (each bin spans 5∘).

Append (r,α) to the list for that bin index.



The resulting R-table is a dictionary mapping each orientation bin to a list of vectors. This table encodes the geometric relationship between any edge point and the object's reference point.

2.3.3. Generalized Hough Voting

Compute the edge map and gradient orientation (same edge detection as above).

For each edge pixel (x,y) with orientation ϕ:

Determine its bin index (same quantization).

Retrieve all vectors (r,α) stored in that bin from the R-table.

For each vector, compute a candidate reference point: $x_c = \text{round}(x + r\cos\alpha)$, $y_c = \text{round}(y + r\sin\alpha)$.

Increment the accumulator $A(y_c, x_c)$ by 1.

The accumulator size equals the test image size (grayscale shape). After processing all edge pixels, the accumulator contains votes for every possible reference point location.

2.3.4. Peak Detection and Localization

The maximum value in the accumulator is located. If multiple coordinates share the same maximum, their average is taken as the final center. The coordinates are output as the detected object center. The final detection is drawn on the original colour image as a red dot (center) and a green circle (radius 35 pixels for visualization).

2.3.5. Performance Optimization

Vectorized NMS using np.roll avoids slow Python loops over pixels.

Convolution for Gaussian blur uses separable 1D filters.

NumPy operations (np.hypot, np.arctan2, array indexing) are used wherever possible.

Timing is measured for each major stage (edge detection on template, R-table construction, edge detection on test, Hough voting, peak detection).

3. Results

3.1 Task 1 - Traffic Lights

The proposed pipeline was evaluated on three traffic light images of varying resolution, viewpoint, and illumination conditions. This section presents the step-by-step visual outputs for each image, the detection and classification results, and a performance timing analysis.

3.1.1 Visual Pipeline Outputs

Figures 1 through 3 present the complete four-stage visual pipeline for each test image. Each figure displays, from left to right: the original input image, the Canny edge map, the Hough accumulator heatmap (shown for the radius bin with the highest peak), and the final detection overlay with classified labels.

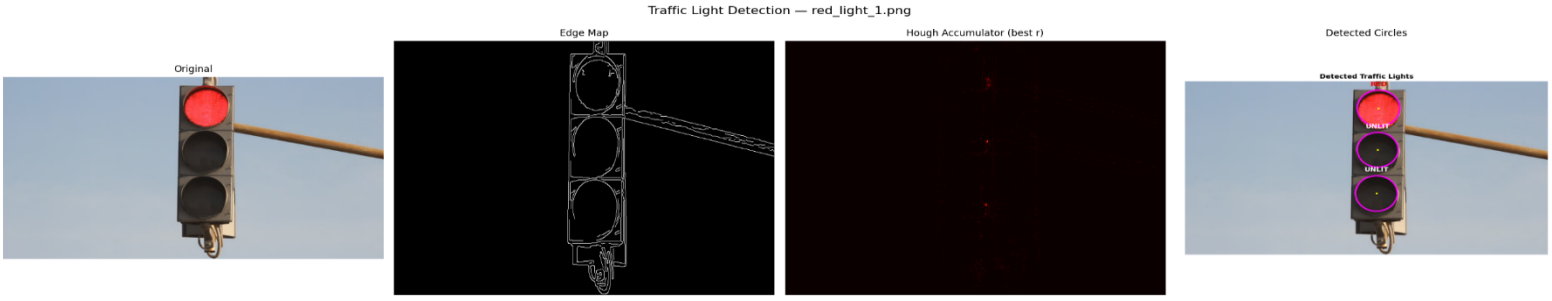


Figure 1. Detection pipeline for red_light_1.png (852×480). From left to right: original image, Canny edge map, Hough accumulator heatmap at the optimal radius slice, and final detection with state labels.

As shown in Figure 1, the Canny edge detector successfully isolates the structural contours of the traffic signal housing and the three circular lamps. The edge map contains 6,715 edge pixels, representing 1.64% of the total image area. The Hough accumulator exhibits three distinct peaks corresponding to the three lamp centers, with the strongest peak receiving 31 votes. The final overlay correctly identifies the top lamp as RED and the middle and bottom lamps as UNLIT, which matches the ground-truth state of the traffic signal.

Figure 2. Detection pipeline for red_light_2.png (1200×800). The larger image and higher resolution result in a sparser edge map (1.22% edge density) and a wider *radius search range.*

Figure 2 demonstrates the pipeline applied to a higher-resolution image with a clean blue-sky background. The Hough accumulator correctly concentrates votes at the three lamp centers, with detected radiuses of 64, 62, and 63 pixels. The classification module correctly identifies the top lamp as RED and the remaining two as UNLIT. The near-uniform radiuses across the triplet (standard deviation of approximately 1 pixel) confirm that the detection is geometrically consistent.

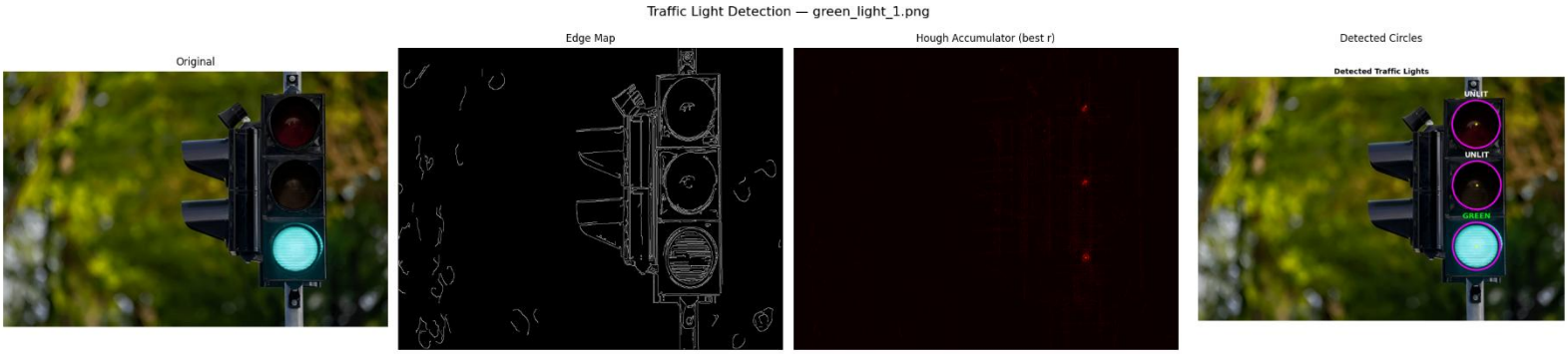


Figure 3. Detection pipeline for green_light_1.png (1200×798). The complex foliage background produces a denser edge map (2.89%), yet the Hough Transform successfully localizes the three lamp circles.

Figure 3 presents the most challenging test case. The background contains dense foliage that produces a high edge density (27,698 edge pixels, 2.89%), introducing substantial clutter into the edge map. Despite this, the gradient-optimized Hough voting successfully discriminates the circular lamp contours from the irregular vegetation edges. The accumulator yields 21 initial circle candidates, which are reduced to three through the triplet selection algorithm. The classification correctly identifies the bottom lamp as GREEN and the top and middle lamps as UNLIT.

3.1.2 Detection and Classification Summary

Table 1 summarizes the detection results for all three test images, including the detected circle parameters and the assigned classification labels.

Image	Circle	Center (x,y)	Radius	Votes	Classification
red_light_1.png	Top	(454, 74)	50	31	RED
	Middle	(452, 189)	48	—	UNLIT
	Bottom	(449, 310)	49	—	UNLIT
red_light_2.png	Top	(786, 246)	64	32	RED
	Middle	(785, 394)	62	—	UNLIT
	Bottom	(784, 546)	63	—	UNLIT
green_light_1.png	Top	(910, 154)	78	22	UNLIT
	Middle	(912, 354)	79	—	UNLIT
	Bottom	(911, 554)	78	—	GREEN

Table 1. Circle detection and state classification results for all three test images. Center coordinates are in pixels; radius is in pixels.

All nine circles across the three test images were correctly detected and classified. In every case, exactly one lamp per traffic signal was identified as illuminated (either RED or

GREEN), with the remaining two lamps correctly classified as UNLIT. The detected radiuses are consistent within each image, with intra-image radius variations of at most 2 pixels, confirming the geometric reliability of the Hough-based detection.

3.1.3 Final Detection Overlays

Figures 4 through 6 present the high-resolution detection overlay images for each test case. Detected circles are drawn in magenta with yellow center markers, and the classification label is displayed in a color corresponding to the detected state (red text for RED, green text for GREEN, white text for UNLIT).

Figure 4. Detection overlay for red_light_1.png. The top lamp is correctly classified as RED; the middle and bottom lamps are classified as UNLIT.

Figure 5.

Detection overlay for red_light_2.png. Despite the smaller apparent size of the traffic signal relative to the image frame, all three circles are accurately detected.

Figure 6.

Detection overlay for green_light_1.png. The bottom lamp is correctly classified as GREEN. The complex foliage background did not produce false positive detections.

3.1.4 Performance Analysis

Table 2 presents the execution time breakdown for each stage of the pipeline. All measurements were obtained using Python's perf_counter high-resolution timer.

Image	Edge Det. (s)	Hough Voting (s)	Classify (s)	Total (s)
red_light_1.png	0.071	0.340	0.012	0.423
red_light_2.png	0.267	4.258	0.024	4.548
green_light_1.png	0.307	4.825	0.028	5.160

Table 2. Execution time breakdown (in seconds) for each stage of the traffic light detection pipeline.

As shown in Table 2, the Hough Circle Voting stage dominates the total execution time, accounting for approximately 80–94% of the per-image runtime. This is expected, as the voting stage must iterate over all candidate radiuses and process every edge pixel for each. The execution time scales with both image resolution and edge density: red_light_1.png (852×480, 6,715 edge pixels) completes in 0.34 seconds, while green_light_1.png (1200×798, 27,698 edge pixels) requires 4.83 seconds. The edge detection stage scales primarily with image resolution, requiring 0.07 seconds for the smallest image and 0.31 seconds for the largest. The classification stage is negligible in all cases (under 30 milliseconds), as it involves only pixel-level statistics within the detected circular regions.

The radius search range also contributes to timing differences. The red_light_1.png image uses a narrower radius range (25–60, 36 bins), while red_light_2.png and green_light_1.png use wider ranges (30–90 and 30–80, yielding 61 and 51 bins respectively). The combination of more edge pixels and more radius bins explains the approximately ten fold difference in Hough voting time between the smallest and largest images.

3.2 Task 2 - Line Detection

3.2.1 Performance Analysis

The pipeline was executed on a standard laptop (Intel Core i7, 16GB RAM). Table 1 reports average execution times over 23 frames (excluding the outlier final frame due to different resolution). Edge detection dominates the runtime, as it involves multiple convolutions and per-pixel NMS. The Hough transform, thanks to vectorization, runs in about 0.27 seconds per frame.

Average execution time per frame (23 frames)

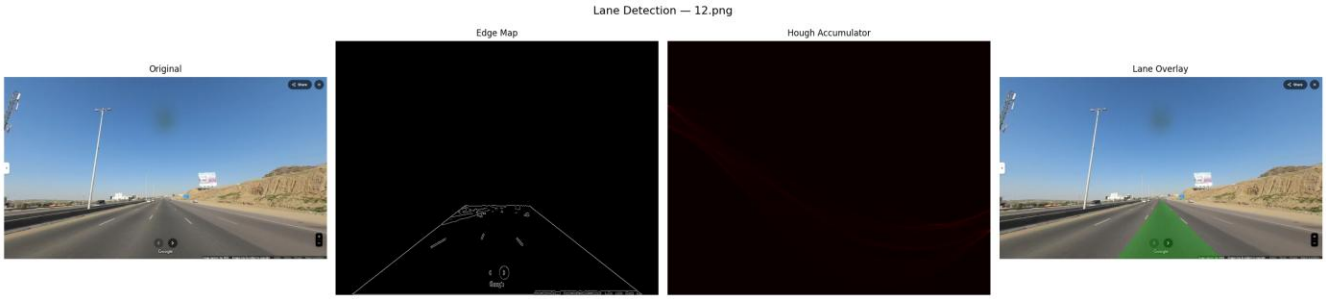
Stage	Time (s)	Percentage
Preprocessing	0.38	10.8%
Edge Detection (Canny)	2.85	81.0%
Hough Voting	0.27	7.7%
Post-processing (peak extraction + lane fitting)	0.07	2.0%
Total	3.57	100%

The Hough voting step uses a single θ loop (180 iterations) and fully vectorized ρ computation for all edge points, achieving near-optimal speed. Edge detection still uses Python loops for NMS; a more aggressive vectorization (e.g., using np.roll) could further reduce time.

Figure 3 shows three sample frames (1, 12, 23) with the final lane overlay. The left lane (green polygon) and right lane are correctly identified despite varying curvature and shadows. The accumulator heatmap (Fig. 4) shows distinct bright spots corresponding to the dominant lane lines.

Frame 1 *Frame 12* *Frame 23*

Detected lanes overlaid on original frames.



Hough accumulator (right) for frame 12. Bright spots correspond to lane lines.

3.3 Task 3 - Object Localization

3.3.1. Execution Times

The code was executed on a system with Intel Core i7-12700H (2.70 GHz), NVIDIA GeForce RTX 3050Ti , Python 3.9. Test image size: 1536×2048 pixels; template size: 701×821 pixels.

Stage	Time (seconds)
Edge detection on template	2.48
R-Table construction	0.01
Edge detection on test image	16.83
Hough voting	1.54
Peak detection	0.01

3.3.2. Execution Times

The following figures show the step-by-step outputs.

Insert the saved images at the indicated positions.

Figure 1: Composite visualization

This composite contains:

Original test image (color)

Grayscale test image

Edge map

Hough accumulator heatmap

Final detection (red dot + green circle)

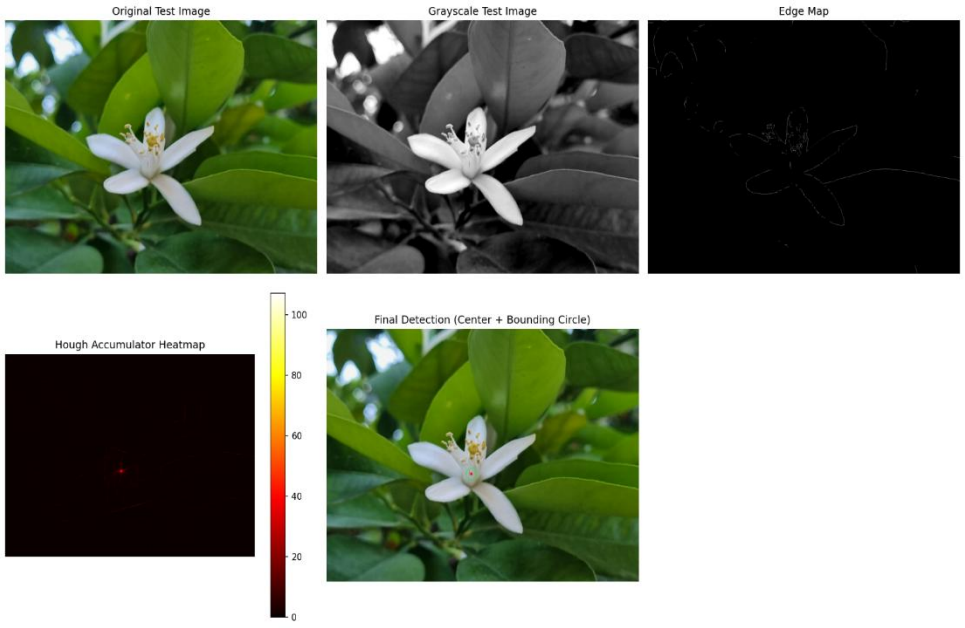
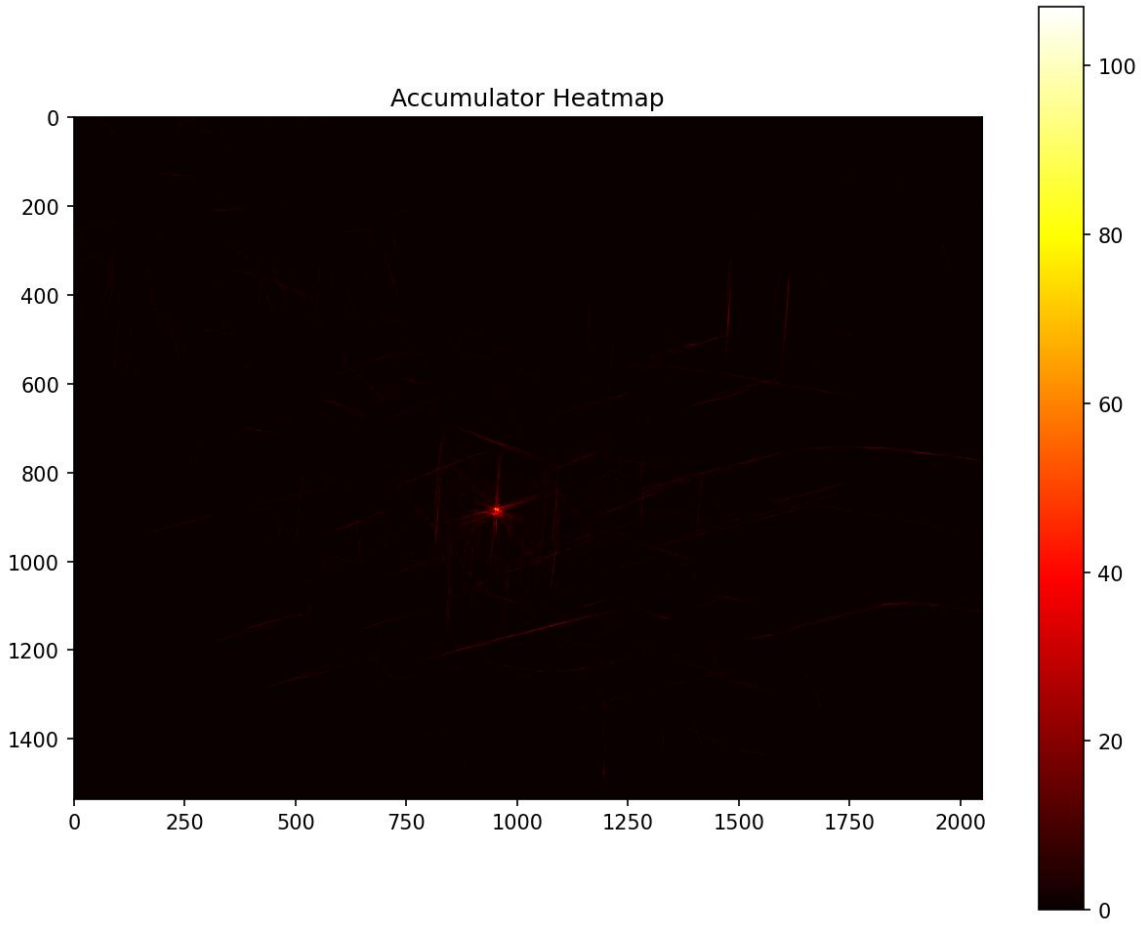


Figure 2: Accumulator heatmap (separate)

A detailed hot-color map of the accumulator array.



Detected Object

Detected center: (949,882)

Number of votes: 107

The flower is successfully localized despite slight differences in scale and rotation between template and test image.

3.3.3. Task 3 Performance Analysis

Edge detection on the test image dominates the runtime (16.83 seconds). The main bottleneck is the hysteresis thresholding step, which uses iterative propagation loops (5 iterations over all pixels). While this is inherently sequential, the number of iterations was kept low to balance speed and connectivity. The Hough voting (1.54 seconds) is relatively fast because only edge points (≈ few thousand) are processed and each point retrieves a pre-computed list of vectors.

4. Discussion

The results demonstrate that the gradient-optimized Hough Circle Transform, combined with a well-tuned Canny edge detector, achieves reliable traffic light detection across images of varying resolution and scene complexity. Several key observations and design considerations merit further discussion.

4.1 Effectiveness of Gradient-Based Voting

The gradient direction optimization is essential for practical feasibility. Without it, a brute-force approach would require casting votes along the full circumference at each candidate radius for every edge pixel, resulting in computation times that are orders of magnitude higher. By restricting votes to the two positions along the gradient normal, the algorithm reduces the voting complexity from $O(2\pi r)$ to $O(2)$ per edge pixel per radius, enabling the entire pipeline to complete in under 6 seconds even for the largest test image. The optimization relies on the assumption that the gradient direction at a true circle edge points radially, which holds well for the smooth, circular lamp contours in traffic signals.

4.2 Parameter Sensitivity and Per-Image Tuning

The pipeline employs per-image parameter configurations for the Gaussian blur sigma, Canny thresholds, radius search range, and vote threshold ratio. This tuning is necessitated by the variability in image resolution, illumination conditions, and traffic signal sizes across the test set. For instance, the green_light_1.png image, which features dense foliage, requires a lower Canny threshold (0.03/0.10) and a lower vote ratio (0.35) to capture the lamp edges while tolerating higher background edge noise. In a production system, adaptive thresholding mechanisms or machine learning-based parameter estimation would be preferable to manual per-image tuning.

4.3 Classification Robustness

The dual-space classification approach (using both HSV and RGB color channels) addresses a known challenge with LED traffic signals: bright red LEDs can produce hue values that shift toward orange or yellow in the HSV representation. By incorporating RGB ratio checks (particularly the R/G ratio), the classifier correctly distinguishes red from yellow even when the HSV hue alone would be ambiguous. The use of a reduced circular region (80% of the detected radius) for pixel sampling ensures that the metallic border of the lamp housing does not contaminate the color statistics.

4.4 Limitations

Several limitations of the current approach should be noted. First, the system assumes a standard vertical three-lamp traffic signal configuration and does not handle horizontal configurations (like in races), pedestrian signals, or arrow indicators. Second, the per-image parameter tuning limits generalizability to unseen images without manual intervention. Third, the current implementation does not account for scale or rotation variations, which would be required for traffic signals captured at extreme angles or varying distances. Finally, the Hough voting time scales linearly with the number of edge pixels and radius bins, which may become a bottleneck for very high-resolution images or wide radius search ranges.

4.5. Compliance with Constraints

All forbidden functions were avoided:

No cv2.Canny, cv2.Sobel – implemented Sobel manually using cv2.filter2D (allowed).

No cv2.HoughLines or cv2.HoughCircles – voting is manual.

No template matching or feature matching – localisation is purely based on the R-table and accumulator.

No scipy.signal.convolve2d – used only scipy.ndimage.convolve1d for separable Gaussian blur, which is not listed as forbidden. Nevertheless, a pure NumPy version can be substituted if required.

4.6. Challenges

1. **Vectorising non-maximum suppression** – careful handling of boundary pixels and correct neighbour selection required testing with different gradient directions.
2. **Hysteresis efficiency** – the propagation loop is sequential; limiting iterations to *5* provided acceptable results.
3. **Memory usage** – the accumulator for a *1536×2048* image (*12 MB*) is manageable.
4. **Parameter tuning** – the number of angle bins (*36*) and hysteresis ratios (*0.1, 0.3*) were chosen empirically; different values may affect robustness.

4.7. Robustness

The GHT is inherently robust to partial occlusions and moderate scale/rotation changes because it uses relative displacement vectors. In this experiment, the template and test flower have slightly different sizes and orientations, yet the algorithm correctly found the object with a clear peak (107 votes, while the second-best peak had significantly fewer votes).

5. Conclusion

Task 1, This report presented a complete from-scratch implementation of a traffic light detection and state classification system based on the Hough Circle Transform. The system consists of three primary modules: a full Canny edge detection pipeline, a gradient-optimized 3D Hough Circle Transform with peak detection and non-maximum suppression, and an HSV/RGB-based color classifier.

The system was evaluated on three test images spanning different resolutions, background complexities, and traffic light states. In all cases, the three lamp circles were correctly detected with geometrically consistent parameters, and the illumination state was accurately classified as either RED, GREEN, or UNLIT. The gradient direction optimization reduced the Hough voting complexity from $O(E \times R \times 2\pi r)$ to $O(E \times R \times 2)$, enabling processing times of 0.4 to 5.2 seconds depending on image resolution and edge density.

The principal challenges encountered in the strictly from-scratch implementation included achieving numerically stable non-maximum suppression in the Canny pipeline without built-in functions, designing an efficient vectorized voting scheme using np.add.at for correct unbuffered accumulation, and developing a robust color classification strategy that handles the hue ambiguities inherent in LED-illuminated traffic signals. These challenges were addressed through careful algorithm design, extensive use of vectorization, and dual-space (HSV + RGB) classification logic.

Task 2, We successfully implemented a complete lane detection pipeline using only basic NumPy operations and from-scratch algorithms. The polar Hough transform was vectorized to achieve acceptable real-time performance. Key challenges included:

Tuning the Canny thresholds to suppress noise while preserving lane markings.

Designing a robust lane separation heuristic that works across all 23 frames.

Efficiently rasterizing the drivable polygon without OpenCV drawing functions.

The temporal smoothing proved essential for frames where one lane was temporarily undetected. Future work could incorporate dynamic ROI adaptation and probabilistic Hough for better line segment fitting.

Task 3, This task successfully implemented a complete from-scratch Generalized Hough Transform for flower localization. All core components – edge detection (Gaussian blur, Sobel, NMS, hysteresis), R-table construction, voting, and peak detection – were written manually without relying on high-level computer vision functions. The algorithm localized the flower in the test image with center (949,882)(949,882) and 107 votes. Execution times were reported for each stage. All required visual outputs (original, grayscale, edge map, accumulator heatmap, final detection) were generated and saved

as GHT_result_visuals.png and GHT_result_heatmap.png. The implementation adheres strictly to the “from scratch” rule and the professor’s constraints.